

LPC21xx CAN Bus Automotive Project: Deep Dive Documentation

This document provides a comprehensive, line-by-line analysis of the automotive CAN bus project featuring three nodes: Main Node, Reverse Alert Node, and Window Glass Control Node. It covers code significance, alternative approaches, bug prevention, and interview questions.

1. System Architecture & Node Overview

The system is distributed across three LPC21xx microcontrollers communicating via the CAN protocol at 125 kbps.

- **Main Node:** Central dashboard. Manages UI (LCD), reads user inputs via Vectored Interrupt Controller (VIC), samples engine temp (DS18B20), and orchestrates commands.
- **Reverse Alert Node:** Activated by the Main Node. Samples distance via an IR/Ultrasonic sensor using the built-in 10-bit ADC, applies digital filtering, and streams telemetry back via CAN.
- **Window Glass Control Node:** Listens for window commands. Manages an array of LEDs representing window position (0 to 8) and provides position feedback.

2. Detailed Code Explanation (Line-by-Line)

2.1 Main Node: Interrupts (EINT.c)

```
volatile u8 flag_win_up = 0;
void __irq EINT0_ISR(void) {
    flag_win_up = 1;
    EXTINT = (1 << 0);
    VICVectAddr = 0;
}
```

Significance: The `volatile` keyword is crucial. It tells the compiler that `flag_win_up` can change outside the normal program flow (inside an ISR). Without it, the compiler's optimizer might cache the value in a register, causing the main loop to never see the flag change and resulting in a dead

button. `_irq` is a compiler directive instructing the compiler to save and restore context (registers) suitable for an interrupt. `EXTINT` clears the hardware interrupt flag so it doesn't fire again immediately. `VICVectAddr = 0;` acknowledges the interrupt to the Vectored Interrupt Controller, allowing lower priority interrupts to fire.

Alternative Code: Instead of setting a flag, the ISR could send the CAN message directly. However, sending CAN messages inside an ISR is highly discouraged as it blocks the processor. Setting a flag (Deferred Interrupt Processing) is the industry standard.

2.2 Main Node: CAN Initialization (CAN.c)

```
PINSEL1 &= ~((3 << 16) | (3 << 18));  
PINSEL1 |= ((1 << 16) | (1 << 18));  
C1BTR = 0x001C001D; /* 125 kbps */
```

Significance: P0.24 and P0.25 are multiplexed. We configure them as TD1 (Transmit) and RD1 (Receive). `C1BTR` configures the CAN bit timing. At 60MHz, a prescaler of 30 and 16 time quanta gives exactly 125 kbps.

Bugs & Fixes: A common bug here is setting the baud rate incorrectly between nodes. If Node A is 125 kbps and Node B is 250 kbps, the bus will collapse into Error Active/Passive states. Always ensure exact BTR matches.

2.3 Reverse Node: ADC Filtering (distance_sensor.c)

```
filtered_distance = (0.15f * raw_dist) + (0.85f * filtered_distance);
```

Significance: This is a Software Exponential Moving Average (EMA) Low-Pass Filter. Sensors are noisy. Taking raw data causes display flickering. Here, 15% of the new reading is added to 85% of the historical reading, creating a smooth transition.

Alternative Code: A simple moving average array could be used (adding last 10 values and dividing by 10), but an EMA filter uses far less RAM (no arrays required) and is faster to execute.

2.4 Main Node: DS18B20 Temperature Sensor

```
DS18B20_WriteByte(DS18B20_CMD_CONVERT_T);  
delay_ms(750);
```

Significance: The DS18B20 uses the 1-Wire protocol. It requires up to 750ms to perform a 12-bit analog-to-digital temperature conversion. If you read before this time, you will get the power-on reset value of 85.0°C.

3. Software and Hardware Bugs & Countermeasures

Category	Possible Bug	Countermeasure / Prevention
Software	Infinite Polling Loops (e.g., <code>while(pin == 0);</code>)	Never use blocking loops for user input. Use state machines, timeout watchdogs, or edge-triggered Interrupts (VIC).
Software	Missing <code>volatile</code> on ISR flags	Always declare variables modified in ISRs and read in <code>main()</code> as <code>volatile</code> .
Hardware	Switch Bouncing	Use software debouncing (e.g., wait 20ms and read again) or hardware RC filters (Resistor-Capacitor network).
Hardware	CAN Bus Reflection / Errors	Ensure 120-ohm termination resistors are placed at both extreme physical ends of the CAN bus.
Hardware	Floating Input Pins	Always enable internal pull-ups or use external 10k resistors on switch inputs to prevent false triggering from EMI.

4. Viva and Interview Questions

Q1: What is the difference between Polling and Interrupts?

Answer: Polling wastes CPU cycles continuously checking a register. Interrupts allow the CPU to sleep or perform other tasks, waking up the CPU instantly via hardware only when an event occurs. Interrupts provide deterministic real-time response.

Q2: Why did we use `C1BTR = 0x001C001D` for 125 kbps? Explain the math.

Answer: With a 60MHz PCLK, the prescaler (BRP) is set to $29+1=30$. So the CAN clock is $60\text{MHz}/30 = 2\text{MHz}$. A bit time consists of 16 time quanta (1 Sync + 13 TSEG1 + 2 TSEG2). Therefore, 1 bit = $16 * (1/2\text{MHz}) = 8$ microseconds. Baud rate = $1/8\mu\text{s} = 125,000$ bps.

Q3: What is the purpose of `VICVectAddr = 0;` at the end of an ISR?

Answer: It clears the vector address register in the ARM7 Vectored Interrupt Controller. This acts as an End of Interrupt (EOI) signal, telling the priority hardware that the current ISR is finished so lower priority interrupts can now be serviced.

Q4: Why does the DS18B20 require a 750ms delay? What happens if you don't wait?

Answer: 750ms is the maximum time required for the sensor's internal ADC to complete a 12-bit conversion. Reading prematurely yields stale data or the default 85°C .

Q5: What is CAN arbitration and how does it work?

Answer: CAN uses Non-Destructive Bitwise Arbitration (CSMA/CD+AMP). If two nodes transmit simultaneously, they monitor the bus. The node transmitting a '1' (Recessive) while the bus is pulled to '0' (Dominant) by another node will lose arbitration, stop transmitting, and become a receiver. Thus, lower ID messages have higher priority.

5. Important Key Points to Remember

- **Separation of Concerns:** Interrupts should be as short as possible (set a flag and exit). The heavy lifting (LCD updates, CAN Tx/Rx delays) must be handled in the main loop.
- **Robustness:** The system implements ECU Fault Flags. If a CAN node disconnects, the main node utilizes a timeout mechanism (500ms watchdog) to detect the missing heartbeat and display a fault, ensuring fail-safe operation.
- **Signal Integrity:** Filtering ADC noise using moving averages and Schmitt-trigger deadbands prevents UI flickering and logic chattering in edge-case physical boundaries.